

1 · Schutzziele & Bedrohungen (STRIDE)

Schutzziel	Bedrohung / Gegenmaßnahme
Authentizität	Spoofing → Signaturen, Zertifikate
Integrität	Tampering → MAC, AEAD, Hash
Nicht-Abstreitbarkeit	Repudiation → Digitale Signatur
Vertraulichkeit	Information Disclosure → Verschlüsselung
Verfügbarkeit	Denial of Service → Rate-Limiting
Autorisierung	Privilege Escalation → ACLs

- **Kerckhoffs:** System sicher auch wenn alles außer Schlüssel bekannt → kein Security by Obscurity
- **Bedrohung beseitigen** > Schwachstelle absichern. Risiko = Bedrohung × Schwachstelle. Sicherheit = Kosten-Nutzen
- **Angreifermodell:** beschreibt Fähigkeiten des Angreifers (= erlaubte Einsatzumgebung). Strenger = stärker

2 · Kryptographische Grundbegriffe

- **Chiffre** $\Pi=(E,D)$: $D(k, E(k, m)) = m$. **K** Keyspace, **M** Messagespace, **C** Cipherspace
- **Schema** $\Pi=(KGen,E,D)$: KGen randomisiert → $k \in K$; E darf randomisiert sein; D deterministisch.
 $r \leftarrow A(x)$ randomisiert; $y := A(x)$ deterministisch
- **Klassische Chiffren:** Caesar $K=\{3\}$, ROT $K=\{0..25\}$. Problem: **Häufigkeitsanalyse** – PT-Statistik im CT sichtbar.

3 · Perfect Secrecy & One-Time-Pad

- **XOR:** $a \oplus a = 0$, $a \oplus 0 = a$, $a \oplus b \oplus b = a$, selbstinvers, assoziativ, kommutativ
- **OTP:** $K = M = C = \mathbb{B}^n$, k uniform. $E(k, m) = m \oplus k$, $D(k, c) = c \oplus k$

Perfect Secrecy: $\Pr[M = m \mid C = c] = \Pr[M = m]$ für alle $m, c \rightarrow$ CT verrät **nichts** über PT

Orakel-Spiel: kein poly-Zeit-Angreifer gewinnt Unterscheidungsspiel mit $\Pr > \frac{1}{2}$ (Raten = $\frac{1}{2}$)

Schranke: P.S. $\Rightarrow |K| \geq |M|$ (bei $|K| < |M|$: $\exists m$ mit $\Pr[C = c \mid M = m] = 0$)

Shannon-Satz ($|K|=|M|=|C|$): P.S. $\Leftrightarrow$ (i) k uniform und (ii) zu (m, c) genau ein k mit $E_k(m) = c$

- **Bayes:** $\Pr[M=m \mid C=c] = \frac{\Pr[C=c \mid M=m] \cdot \Pr[M=m]}{\Pr[C=c]} \Rightarrow \Pr[E_k(m)=c]$ unabhängig von m
- **OTP:** Schlüssel so lang wie Nachricht; echter Zufall; nie zweimal!

4 · Block- & Streamchiffren

- **AES** (Blockchiffre): Block immer 128 Bit; Schlüssel 128/192/256 Bit. NIST 2001
- **ChaCha20** (Streamchiffre): Keystream aus $(Key+Nonce) \oplus PT$. TLS 1.3 Standard. RC4 = **unsicher**

Nonce-Reuse: Gleiche (Nonce, Key) $\Rightarrow$ gleicher Keystream $\Rightarrow c \oplus c' = m \oplus m'$ (statistisch analysierbar). Verfahren **komplett gebrochen**!

5 · Betriebsmodi (>1 Block)

- **PKCS#7 Padding:** fehlende Bytes mit Wert=Anzahl füllen (1..16). Voller Extrablock wenn $|m| \bmod n = 0$. Letztes Byte = Anzahl. Injektiv.

Modus Verschlüsselung / Eigenschaften

ECB $c_i = E(k, m_i)$. Pad: ja. **NIE! Gleiche Blöcke $\rightarrow$ gleiche CT (Pinguin-Angriff)**

CTR $c_i = E(k, N_i) \oplus m_i$, $N_i = N \parallel i$. Pad: nein. Streamcipher-artig, parallelisierbar, nur $E()$ nötig. $N = \text{Nonce}(n \text{ Bit}) \parallel \text{Counter}(r \text{ Bit})$, $n+r = 128$. Max. Blöcke pro (N, K) : 2^r

CBC $c_i = E(k, m_i \oplus c_{i-1})$, $c_0 = IV$. Pad: ja. Sequentiell. $D: m_i = D(k, c_i) \oplus c_{i-1}$

IV/CT-Manipulation (Malleability)

Modus	Bit-Flip in CT	IV-Änderung
ECB	Flip c_i -Bit $\Rightarrow$ exakt selbes PT-Bit flippt (gezielte!)	ganzer Keystream korrupt
CTR	Flip c_{i-1} -Bit $\Rightarrow$ flippt m_i -Bit; m_{i-1} komplett kaputt	nur 1. Block betroffen

CTR & CBC: nur Vertraulichkeit, keine Integrität!
Für aktiven Angreifer $\rightarrow$ AEAD!

Vaudenay Padding Oracle: Angreifer erfährt ob Padding falsch $\rightarrow$ CBC-CT entschlüsselbar. Bsp: ASP.NET, **POODLE** (SSL 3.0).

6 · Angreifermodelle

Modell Fähigkeit / Beispiel

EAV	nur Challenge-CT – ECB bereits hier unsicher
KPA	(m, c) -Paare bekannt – ROT sofort gebrochen
CPA	Verschlüsselungs-Orakel – Midway, Web-Embedding
CCA	Ver- und Entschlüsselungs-Orakel – Router leitet CT

PRF: Angreifer kann $F(k, \cdot)$ nicht von Zufallsfunktion unterscheiden.

AES-CTR/CBC: **CPA-sicher** (falls AES PRF), **nicht CCA** (malleable)

AES-GCM: **CCA-sicher** (AEAD, Manipulation vor Entschlüsselung erkannt)

7 · MAC & Authenticated Encryption

- **MAC** $M_k(m)$: Tag T – schützt **Integrität** + **Authentizität** (keine Vertraulichkeit!). Ziel: kein forgery under chosen-message. Konstruktionen: HMAC, Poly1305, CMAC

AE/AEAD: $E(K, P) = (C, T)$; $D(K, C, T) = P$ oder **error**. Erkennt Manipulation VOR Entschlüsselung $\rightarrow$ CCA-sicher.

AES-GCM: populärstes AEAD. **ChaCha20-Poly1305:** TLS 1.3.

Encrypt-then-MAC: erst CT, dann MAC – alles andere unsicher! Nie selbst implementieren.

8 · Hashfunktionen & Passwörter

Eigenschaft	Definition (Stärke: stark → schwach)
-------------	--------------------------------------

Kollisionsresistenz schwer $x \neq x'$ mit $f(x) = f(x')$

Second-Preimage zu x finde $x' \neq x$ mit $f(x) = f(x')$

Preimage zu y finde x mit $f(x) = y$

- **MD5 (gebrochen 2005), SHA-1 (gebrochen); SHA-2** (SHA-256/384/512); SHA-3; BLAKE2
- Commitment: $H(\text{salt}||b)$ veröffentl., später öffnen. Integritätscheck: $H(\text{Daten})$ sicher übermitteln
- **Geburtstagsparadoxon**: M Werte $\rightarrow \approx \sqrt{M}$ Objekte für Kollision $>50\%$. n -Bit-Hash: Kollision nach $\approx 2^{n/2}$

Passwort-Hashing-Evolution

$H(\text{pw}) \rightarrow$ Rainbow-Table angreifbar
 $\rightarrow H(\text{salt}||\text{pw})$, Salt zufällig pro Nutzer $\rightarrow$ Vorab-DB nutzlos
 $\rightarrow$ HW zu schnell $\rightarrow$ KDF (langsam, memory-hard)

9 · Key-Derivation-Functions (KDF)

Typ	Algorithmen	Zweck
Passwort-KDF	Argon2, bcrypt, scrypt, PBKDF2	langsam, memory-hard, HW-resistent
Schlüssel-KDF	HKDF	schnell; NICHT für Passwörter!

- Argon2-Parameter: **i**=Iterationen, **m**=Speicher, **p**=Parallelismus. Speichern: (salt, Argon2(salt,pw,p.))

10 · Zufallszahlen (RNG)

- **PRNG (nie für Krypto!)**: java.util.Random, Python random() – vorhersagbar, rückrechenbar
- **CSPRNG**: Linux: /dev/urandom, getrandom(); Windows: BCryptGenRandom; Intel rdrand (nur 1 Quelle)
- **Boot-Zeit = wenig Entropie!** HW-RNG nur eine Quelle von mehreren

Fortuna (FreeBSD): Zustand $S = (C, K)$. GenerateBlocks: $r || = E_K(C)$; $C++$. Key-Rekeying $K \leftarrow \text{GenerateBlocks}(S, 2)$ nach Output.

Backtracking: aus K_i nicht K_{i-1} berechenbar (Vergangenheit geheim)

Forward Security: neuer Zustand bei unbekanntem K uniform

11 · RSA

- Setup: p, q prim; $N = p \cdot q$; $e \cdot d \equiv 1 \pmod{(p-1)(q-1)}$. Public (e, N) , Private (d, N)
- Enc: $c = m^e \bmod N$. Dec: $m = c^d \bmod N$. Korrektheit: $c^d = m^{ed} = m$ (Euler/Fermat)

Rechenrezept: Square-and-Multiply – nach jedem Schritt mod N !

$x \bmod N = x - \lfloor x/N \rfloor \cdot N$.

Bsp: $N=187, e=3, m=11$: $11^3=1331 \bmod 187 = 1331 - 7 \cdot 187 = 22$

DH-Bsp: $N=17, g=3, a=3$: $A = 3^3=27 \bmod 17 = 10$.

$B=9$: $K = 9^3=729 \bmod 17 = 15$

Textbuch-RSA ohne Padding unsicher! Praxis: RSA-OAEP (Verschlüsselung), RSA-PSS (Signatur)

12 · Diffie-Hellman & ECC

- **DH**: N prim, g Generator. Alice: $a \leftarrow \text{rand}$, schickt $A = g^a \bmod N$. Bob: $b \leftarrow \text{rand}$, schickt $B = g^b \bmod N$. Gemeinsam: $K = B^a = A^b = g^{ab} \bmod N$
- Sicherheit: Diskreter Logarithmus (a aus $g^a \bmod N$) schwer

DH ohne Auth $\rightarrow$ MITM! Eve macht 2 separate Austausch. Fix: Signaturen + Zertifikate (ECDHE-Variante in TLS)

- **ECC**: ≈ 256 Bit ECC $\approx$ RSA-3072. Schneller, kleinere Schlüssel
- **Quanten/Shor**: bricht RSA, DH, ECC. Q-Day ≈ 2029 . **Post-Quantum**: ML-KEM (Kyber, KEX), ML-DSA (Dilithium, Sig.)
- **PFS**: ephemere (EC)DHE-Schlüssel nach Session gelöscht $\rightarrow$ aufgezeichneter Traffic nicht entschlüsselbar. „Harvest now, decrypt later“

13 · Digitale Signaturen

- $\text{Sign}_{\text{priv}}(m) = \sigma$; $\text{Verify}_{\text{pub}}(m, \sigma) = \text{true/false}$
- Gibt: **Authentizität + Integrität + Nicht-Abstreitbarkeit**
- RSA-PSS, ECDSA. Sicher unter EUF-CMA. In Praxis: $\text{Sign}(H(m))$ (für große Nachrichten)

14 · PKI & X.509-Zertifikate

- **Zertifikat**: Subject (Identität) an Public Key gebunden, signiert vom Issuer. Felder: Subject, Issuer, Valid From/Until, PublicKey, Extensions (CA-Flag, pathlen), Signatur

Kette: Blatt(Server) $\rightarrow$ Zwischen-CA $\rightarrow$ Root-CA

Root: Subject=Issuer (self-signed). Issuer[i] = Subject[i+1]

Gültigkeitsprüfung – alle 4 Bedingungen:

1. **Signaturen** jedes Zertifikats verifizieren (mit Public Key des Issuers)
2. **Zeit**: Valid From $\leq$ jetzt $\leq$ Valid Until für jedes Zertifikat
3. **Constraints**: CA=true für Zwischen-CAs; pathlen= $k \rightarrow$ max. k CAs unterhalb
4. **Trust Anchor**: Root-CA im Client Trust Store (vorkonfiguriert!)

15 · Kerberos

- **KDC** = AS (Auth. Server) + TGS (Ticket Granting Server). Kennt Langzeit-Schlüssel aller Nutzer/Dienste
- Login $\rightarrow$ TGT vom AS $\rightarrow$ Service-Ticket vom TGS $\rightarrow$ Dienst

KDC kompromittiert: kennt alle Langzeit-Schlüssel $\rightarrow$ kann beliebige Tickets ausstellen $\rightarrow$ jede Identität vortäuschen. KDC = Single Point of Trust/Failure!

16 · Human Authentication (Passwörter & 2FA)

- **Speichern:** nie Klartext! → (salt, KDF(salt, pw)) mit zufälligem Salt pro Nutzer; KDF = Argon2/bcrypt
- **Faktoren:** Wissen (PW), Besitz (Token), Inhärenz (Bio). 2FA = mind. 2 verschiedene Faktoren
- **TOTP:** Codes nur einmal nutzbar. $\text{Sign}_{\text{priv}}(\text{Challenge})$ beweist Key-Besitz.

17 · Netzwerksicherheit – Grundangriffe

Angriff	Beschreibung / Gegenmaßnahme
Spoofing	falsche Identität → Signaturen, Zertifikate
MITM	dazwischensetzen → AEAD + Auth.
Replay	Msg erneut einspielen → Nonce/Timestamp/Seq.
Reflection	Antworten umlenken/verstärken → asym. Proto.
DoS	Verfügbarkeit → Rate-Limiting, Puzzle
Downgrade	schwächere Version erzwingen → Transcript, HSTS

Replay-Analyse: Nonce/Timestamp muss von **jedem** Partner frisch erzeugt und im MAC **gebunden** sein. DH+Sign schützt Schlüssel, aber nicht den Austausch selbst → Replay möglich, gleicher K_0 !
PFS-Update: $K_{i+1} = H(K_i)$ nach jeder Msg → Forward Secrecy

18 · TLS 1.2 – Handshake

C→S: ClientHello(Ver, Random_C, SessionID, CipherSuites[, SessionTicket])
S→C: ServerHello(Ver, Random_S, Suite)
S→C: Certificate
S→C: ServerKeyExchange(g,N,g^a; Sig.) [bei DHE]
S→C: ServerHelloDone
C→S: ClientKeyExchange(g^b) [PMS=g^{ab}]
C→S: ChangeCipherSpec · Finished
S→C: ChangeCipherSpec · Finished
MS=PRF(PMS, "master secret", RC||RS) → 4 Session-Keys

- **Ciphersuite (1.2):** KEX-Auth-Cipher-MAC. Bsp: ECDHE-RSA-AES256-GCM-SHA384
- **Session-Ticket:** im ClientHello in Extension session_ticket. Akzeptiert wenn Server mit ServerHello + ChangeCipherSpec + Finished antwortet (kein Cert/SKE = abgekürzter Handshake)
- **Gewählte Suite:** steht im **ServerHello**

19 · TLS 1.3 – Vergleich

Aspekt	TLS 1.2 vs. TLS 1.3
Handshake-RTT	1.2: 2 RTT, RSA oder (EC)DHE 1.3: 1 RTT, nur (EC)DHE (PFS immer)
Ciphersuite	1.2: KEX-Auth-Cipher-MAC 1.3: TLS_<AEAD>_<HASH> (2 Teile, nur AEAD)
Handshake-Auth	1.2: ServerKeyExchange+Sig 1.3: CertificateVerify (Sign. über Transcript)
Resumption	1.2: Session-ID/Ticket 1.3: PSK (Pre-Shared Key)

TLS 1.3 Ciphersuites: TLS_AES_256_GCM_SHA384 · TLS_CHACHA20_POLY1305_SHA256 · TLS_AES_128_GCM_SHA256
CertificateVerify: Server signiert Transcript mit priv. Key → (1) Authentizität, (2) Integrität (kein Downgrade), (3) Zertifikat an KEX gebunden
Finished: PRF über alle HS-Nachrichten – schützt vor nachträglicher Manipulation

PFS: ephemere (EC)DHE-Werte gelöscht → Key-Diebstahl gibt Traffic nicht preis. „Harvest now, decrypt later” → PFS schützt. **E2E:** TLS nur bis Server (kein E2E mit Proxy!); Signal/WhatsApp: echtes E2E.

21 · Downgrade-Angriffe

- Angreifer manipuliert ClientHello → streicht starke Suites → Server wählt schwache Suite
- **POODLE:** SSL 3.0 Fallback erzwingen → Padding Oracle auf CBC
- **FREAK:** schwache RSA-Exportkeys erzwingen. **SMACK/SLOTH:** MD5 in TLS erzwingen
- **Fix TLS 1.3:** CertificateVerify über Transcript → unbemerkte Modifikation unmöglich

22 · Protokoll-Sicherheitsanalyse – Rezept

- Schutzziele:** Vertraulichkeit, Integrität, Auth., Replay, PFS?
- Primitive:** CPA oder CCA? AEAD? Signatur? MAC über was?
- Angriffsvektoren:**
 - Replay? → fehlt Freshness (Nonce im MAC gebunden?)
 - MITM? → fehlt Auth. des KEX?
 - Downgrade? → Transcript gebunden?
 - Malleability? → CTR/CBC ohne MAC?
 - Nonce-Reuse? → Wie wird Nonce gewählt?
- Fix:** AEAD statt CPA; Nonce für Freshness; Sign für Auth.; Transcript-Hash gegen Downgrade

Problem	Symptom / Fix
Keine Integrität	CTR/CBC, Bit-Flip im CT → AES-GCM
Replay	gültige Msg erneut akzeptiert → Nonce/Seq. im MAC
MITM bei KEX	DH ohne Auth → Signaturen + Zertifikate
Padding Oracle	CBC + Fehler-Info → Encrypt-then-MAC; AEAD
Nonce-Reuse	gleiche (K, N) → Counter-Nonce erzwingen
Downgrade	ClientHello manipuliert → Transcript signieren CRIME/BREACH: CT-Länge verrät PT
Kompression+I	→ kein Komprimieren wenn Angreifer PT beeinflusst
PRNG statt CSPRNG	vorhersagbare Keys → /dev/urandom

23 · Schnell-Entscheidungen	
Ziel	Verfahren
Vertraulichkeit (passiv)	AES-CTR oder AES-CBC
Vertr. + Integrität (aktiv)	AES-GCM / ChaCha20-Poly1305
nur Integrität	HMAC / Poly1305 / CMAC
Schlüsselaustausch (PFS)	ECDHE / DHE → danach auth.!
Authentifizierung / Signatur	RSA-PSS / ECDSA + PKI
Passwort speichern	Salt + Argon2 (nie $H(\text{pw})$ allein!)
Krypto-Zufallszahlen	<code>/dev/urandom</code> ; <code>BCryptGenRandom</code>
Hash / Integritätscheck	SHA-256/384/512
NIE: ECB · MD5/SHA-1 für Sig. · RC4 · Nonce/Schlüssel doppelt · <code>java.util.Random</code> / <code>random()</code> für Krypto · OTP-Schlüssel wiederverwenden · Krypto selbst implementieren · RSA ohne Padding · DH ohne Authentifizierung	

25 · Klausur-Schnellreferenz	
Aufgabentyp	Antwort
Schema X gg. Y sicher?	CTR/CBC → CPA (wenn AES PRF), nicht CCA. GCM → CCA. OTP → Perfect Secrecy ($ K \geq M $, k uniform, nie zweimal!)
MAC $M' = M \parallel \text{const}$?	Sicher: abtrennbar, Fälschen erfordert $M(k, m) \rightarrow$ kein Vorteil
Zertifikatskette	Issuer[i]=Subj[i+1]; Root: Subj=Iss; Zeit, pathlen, CA-Flag, TrustStore?
Protokoll-Analyse	1.Schutzziele → 2.Primitive → 3.Freshness/Auth/Transcript? → 4.Fix
Modus & RSA	Aktiv→AEAD; passiv→CTR/CBC; nie ECB. $c = m^e \bmod N$, Square-and-Multiply
DH-MITM	Eve macht 2 Austausche → kennt beide Session-Keys → kein echtes Shared Secret